

TESTARE

SeniorWatch – Strategie și acoperire a testelor automate

Identificarea Documentului

Cod proiect:	MPSAM-2026-AAL-01
Titlu proiect:	SeniorWatch – Sistem de Teleasistență la Domiciliu a Persoanelor în Vârstă
Versiune document:	1.0
Data:	16.06.2026
Document de referință:	Proiectare (v2.0)
Locație cod teste:	tests/

Cuprins

1. [Scop și domeniu](#)
2. [Arhitectura testării în proiect](#)
3. [Teste Web \(Jest\)](#)
4. [Teste Embedded \(Unity + PlatformIO\)](#)
5. [Mapare teste → module Proiectare](#)
6. [Rularea testelor](#)
7. [Acoperire curentă și lacune](#)
8. [Automatizare CI/CD](#)

1. Scop și domeniu

Acest document descrie **strategia de testare automată** a platformei SeniorWatch, în concordanță cu arhitectura din documentul de **Proiectare**. Scopul testelor este validarea logicii critice pe componentele unde există cod executabil testabil fără infrastructură AWS sau hardware ESP32.

Principii:

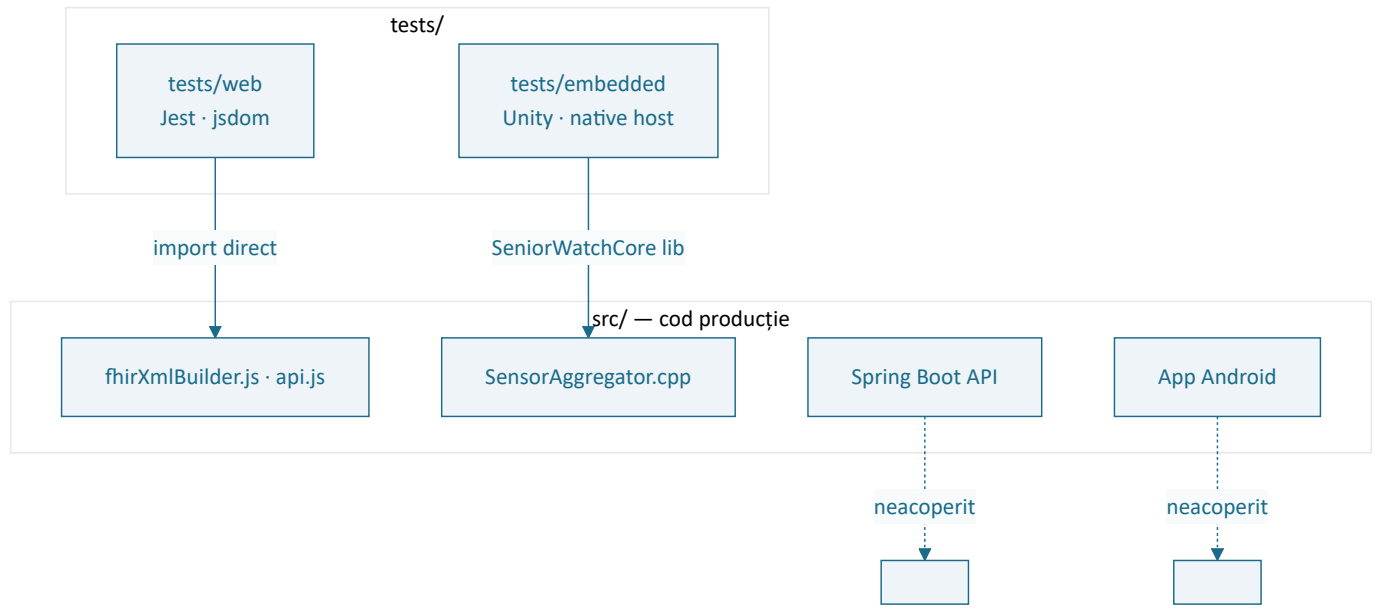
- **Teste unitare** pe cod real din `src/` , nu pe copii duplicate.
- **Izolare de infrastructură:** `fetch` și `localStorage` sunt mock-uite în testele web; firmware-ul rulează pe host (`native`) prin stub-uri Arduino.
- **Tooling separat per ecosistem:** Node.js (Jest) pentru web, PlatformIO + Unity pentru embedded.

Domeniu acoperit în `tests/` :

Suită	Tehnologie	Cod sursă testat
<code>tests/web/</code>	Jest 29, jsdom	<code>fhirXmlBuilder.js</code> , <code>api.js</code>
<code>tests/embedded/</code>	Unity, PlatformIO <code>native</code>	<code>SensorAggregator.cpp</code>

În afara domeniului curent: backend Spring Boot (`src/cloud`), aplicația Android, teste end-to-end UI, teste pe hardware ESP32.

2. Arhitectura testării în proiect



Structura directorului `tests/` :

```

tests/
├── README.md
├── web/
│   ├── package.json
│   ├── jest.config.js
│   ├── babel.config.js
│   └── unit/
│       ├── fhirXmlBuilder.test.js
│       └── api.test.js
└── embedded/
    ├── platformio.ini
    ├── lib/
    │   ├── ArduinoStub/      # millis(), Serial, DHT – stub host
    │   └── SeniorWatchCore/  # leagă SensorAggregator.cpp
    └── test/
        └── test_sensor_aggregator/
            └── test_main.cpp

```

Convenții de denumire:

- Web: `*.test.js` (descoperire automată Jest în `unit/`).
- Embedded: `test_<modul>/test_main.cpp` (convenție PlatformIO Unit Testing).

3. Teste Web (Jest)

3.1. Configurare

Atribut	Valoare
Framework	Jest 29
Mediu	<code>jsdom</code> (simulare browser)
Transform	<code>babel-jest</code>
Pattern	<code>unit/**/*.test.js</code>
Import sursă	<code>../../../../../src/web/src/...</code> (cod producție)

3.2. `fhirXmlBuilder.test.js` — export HL7 FHIR R4

Testează modulul `src/web/src/utils/fhirXmlBuilder.js`, responsabil de generarea **Bundle-ului FHIR R4 (XML)** descris în documentul HL7-FHIR.

Grup	Ce verifică
Bundle structure	declarație XML, tag <code><Bundle xmlns="http://hl7.org/fhir"></code> , calcul <code>total</code> , acceptă listă simplă de pacienți
Patient resource	mapare demografice, identificer CNP (<code>https://hl7.ro/fhir/sid/cnp</code>), gender M/F/unknown

Grup	Ce verifică
Encounter / Condition	Encounter <code>finished</code> , Condition cu cod ICD-10
Observation	puls, SpO ₂ , temperatură, umiditate (LOINC); omitere dacă lipsește <code>latestSample</code>
XML escaping	<code>&</code> , <code><</code> , <code>></code> , <code>"</code> , <code>'</code>

12 cazuri de test în acest fișier.

3.3. `api.test.js` — client REST

Testează `src/web/src/api.js` — wrapper-ele `fetch` către backend-ul Spring Boot (autentificare JWT, CRUD).

Funcție	Comportament verificat
<code>loginUser</code>	POST <code>/api/auth/login</code> , parsare JSON, eroare la 4xx/5xx, body gol → <code>{}</code>
<code>getPatients</code>	header <code>Authorization: Bearer <token></code> din <code>localStorage</code>
<code>createUser</code>	POST <code>/api/users</code> cu payload <code>{email, password, role}</code>
<code>logoutUser</code>	șterge <code>sw_token</code> , <code>sw_role</code> , <code>sw_email</code> din <code>localStorage</code>

`fetch` și `localStorage` sunt mock-uite cu `jest.fn()` / `jest.spyOn()` .

7 cazuri de test în acest fișier.

Total suită web: 19 teste.

4. Teste Embedded (Unity + PlatformIO)

4.1. Configurare

Atribut	Valoare
Framework	Unity (via PlatformIO)
Mediu	<code>native</code> (compilator C++ de host, fără ESP32)
Standard	<code>gnu++17</code> , flag <code>-DUNIT_TEST</code>
Sursă	<code>src/embedded/include/</code> + <code>SensorAggregator.cpp</code>
Stub	<code>lib/ArduinoStub/</code> — <code>millis()</code> , <code>Serial</code> , tipuri Arduino, DHT

`platformio.ini` redirecționează `include_dir` către `src/embedded/include` , iar biblioteca `SeniorWatchCore` compilează **doar** `SensorAggregator.cpp` (fără `main.cpp` firmware).

4.2. `test_sensor_aggregator` — mediere senzori

Testează `SensorAggregator` — componenta care colectează eşantioane la ~1 s şi calculează cadrul mediat la ~10 s (conform Proiectare §1.1).

Test	Verificare
<code>test_average_heart_rate_for_valid_samples</code>	media HR 70/80/90 → 80 bpm; medie temp/umiditate
<code>test_average_filters_out_unphysiological_hr</code>	exclude HR 0 şi 300; media 60+80 → 70
<code>test_env_invalid_when_no_env_samples_present</code>	<code>validReading=false</code> → temp/umiditate 0
<code>test_is_ready_after_ten_seconds</code>	<code>isReady()</code> false <10 s, true la ≥10 s (<code>__testSetMillis</code>)
<code>test_reset_clears_samples</code>	după <code>reset()</code> , medie HR 0, env invalid
<code>test_addSample_caps_at_max_samples</code>	20 eşantioane → media stabilă (cap buffer)

Total suită embedded: 6 teste.

4.3. De ce stub Arduino?

Firmware-ul include `Arduino.h` , `DHT.h` etc., disponibile doar cu toolchain-ul ESP32. Pentru testare pe host, `ArduinoStub` reimplementează minimul necesar. Helper-ul `__testSetMillis(value)` permite avansarea timpului virtual în testele de fereastră.

5. Mapare teste → module Proiectare

Modul (Proiectare §2.2)	Acoperire teste	Observații
ESP32 / <code>SensorAggregator</code>	Da — 6 teste Unity	logică mediere; fără BLE, fără hardware
Web — FHIR export	Da — 12 teste Jest	<code>FhirXmlBuilder</code> ; fără UI React
Web — client API	Da — 7 teste Jest	autentificare, token, logout
Web — UI (React)	Nu	pagini medic/pacient/admin netestate
Cloud (Spring Boot)	Nu	<code>spring-boot-starter-test</code> în <code>pom.xml</code> , fără <code>@Test</code>
Android	Nu	descrie la nivel arhitectură
BLE / ingestie batch	Nu	necesită integrare sau emulator

6. Rularea testelor

6.1. Web (Jest)

```
cd tests/web
npm install
npm test          # o singură rulare
npm run test:watch # mod watch
npm run test:coverage # raport acoperire
```

6.2. Embedded (Unity)

```
cd tests/embedded
python -m platformio test -e native
```

Pe Windows, dacă `pio` nu este în PATH: `python -m platformio` în loc de `pio`.

Cerințe: PlatformIO Core, compilator C++ de host (MSVC/MinGW pe Windows).

6.3. Rezumat comenzi

Componentă	Director	Comandă
Web	tests/web	npm install && npm test
Embedded	tests/embedded	python -m platformio test -e native

6.4. Output așteptat

Web (Jest):

```
Test Suites: 2 total
Tests:       19 total (fhirXmlBuilder: 12, api: 7)
```

Embedded (Unity):

```
===== 6 tests, 0 failures =====
```

7. Acoperire curentă și lacune

7.1. Ce este acoperit

- Generarea corectă a documentelor **FHIR R4 XML** (resurse, terminologii, escape XML).
- Comunicarea **REST autentificată** din portalul web (login, token Bearer, logout).
- **Medierea senzorilor** pe fereastra de 10 secunde, filtrul HR fiziologic (30–220 bpm), reset și cap buffer.

7.2. Lacune cunoscute

Zonă	Descriere
fhirXmlBuilder	1 test în divergență: <code>ignora intrarile fara pacient</code> — așteaptă <code>total=0</code> , implementarea emite Patient gol
Backend Cloud	fără teste JUnit; deploy folosește <code>-DskipTests</code>
UI React	fără teste componentă (<code>@testing-library/react</code> neutilizat în <code>tests/web</code>)
Firmware complet	doar <code>SensorAggregator</code> ; fără BLE GATT, ECG streaming, <code>main.cpp</code>
Android	fără suită în <code>tests/</code>
Integrare E2E	fără teste Cypress/Playwright sau Postman automatizat
CI teste	workflow-urile GitHub (<code>deploy-web.yml</code> , <code>deploy-backend.yml</code>) nu rulează teste

7.3. Direcții recomandate

1. **Backend:** teste JUnit pentru controllere (`/api/auth/login` , `/api/measurements` idempotent pe `batchId`).
2. **Web:** teste componentă pentru paginile `/medic/hl7` și `/admin/adminhl7` .
3. **CI:** workflow `test.yml` care rulează `npm test` și `pio test -e native` la fiecare push pe `master` .
4. **Embedded:** extindere către `BoardManager` / flux BLE după modelul `SensorAggregator` .

8. Automatizare CI/CD

Workflow	Trigger	Rulează teste?
<code>.github/workflows/deploy-web.yml</code>	<code>push master</code> , <code>src/web/**</code>	Nu — doar <code>npm ci + npm run build</code>
<code>.github/workflows/deploy-backend.yml</code>	<code>push master</code> , <code>src/cloud/backend/**</code>	Nu — <code>mvn clean package -DskipTests</code>

Testele se rulează **manual** de dezvoltatori, conform instrucțiunilor din `tests/README.md` și subfolderele `web/` și `embedded/` .

Responsabilitate: fiecare programator rulează testele componentei atribuite înainte de commit pe modulele modificate (conform Proiectare — colectivul de elaborare).